

Кроссмикроконтроллерная кооперативная ОС

Основная суть проекта - абстрагировать бизнес логику от микроконтроллерной платформы. А так же создать систему разработки легковесных приложений на базе микроконтроллеров, с небольшими ресурсами, с переиспользуемыми исходными кодами.

Подготовка к работе

1. Скачать и установить [тулчейн](#)
2. Скачать и установить систему сборки [Ninja](#)
3. Скачать и установить [CMake](#)
4. Скачать и установить NuOpenOCD, ищем все [тут](#)
5. Скачать и установить OpenOCD (TODO: разобраться с Gigadevice)
6. При использовании J-Link скачать и установить [J-Link Software](#)
7. Возможно понадобится добавить в PATH директории тулчейна, Ninja, NuOpenOCD ...

Сборка проекта

```
cd <COS dir>
cmake.exe -D CMAKE_BUILD_TYPE=Debug -D BOARD=<name> -S '.' -B './build' -G
Ninja
cd ./build\
cmake --build . --clean-first
```

В параметре `-D BOARD=` указывается имя платформы. Один проект может быть нацелен на множество платформ. Параметр `-D BOARD=` можно опустить если он указан в `.\Projects\CMakeLists.txt`.

Пример. Прошивка микроконтроллера Nuvoton M463KG

Локально через OpenOCD:

```
$ nuopenocd.exe -s "./HDL/McuPort/ARM/Nuvoton/NUM463KG/Res" -f
"./HDL/McuPort/ARM/Nuvoton/NUM463KG/Res/tool.cfg" -f
"./HDL/McuPort/ARM/Nuvoton/NUM463KG/Res/mcu.cfg" -c "init" -c "halt" -c "flash
write_image erase ./build/bmc.hex" -c "reset run"
```

Где `tool.cfg` - конфиг NuLink,

`mcu.cfg` - конфиг контроллера.

Все можно найти в пакете SDK под конкретную серию микроконтроллера.

Удаленно через GDB + OpenOCD:

```
$ arm-none-eabi-gdb.exe
(gdb) target remote 10.20.30.40:3333
(gdb) monitor reset halt
(gdb) monitor flash erase_address 0x00000000 0x00040000
(gdb) monitor flash info 0
(gdb) monitor flash protect 0 0 7 off
(gdb) file ./build/bmc.elf
(gdb) load
(gdb) monitor reset run
```

Команды `(gdb) monitor flash info 0` и `(gdb) monitor flash protect 0 0 7 off` можно пропустить. Первая необходима чтобы посмотреть состояние защиты флеш памяти от записи, вторая снять защиту.

Аналогично для других поддерживаемых микроконтроллеров. Скрипты можно найти в директории контроллера, в `./Res`.

Установка и настройка среды VS Code

1. Скачать и установить [VS Code](#)
2. Разрешить установить расширения C/C++, CMake, Cortex-Debug... (см. `./.vscode/extensions.json`)

Готово.

Сборка в среде

Для сборки проекта необходимо определить имя платформы `BOARD`. Это можно сделать через пресет `CMakePresets.json` в VisualCode:

```
{
  "name": "XXX_D",
  "displayName": "XXX Debug",
  "generator": "Ninja",
  "binaryDir": "${sourceDir}/build",
  "cacheVariables": {
    "CMAKE_BUILD_TYPE": "Debug",
    "BOARD": "XXX"
  }
},
{
  "name": "XXX_R",
  "displayName": "XXX Release",
  "generator": "Ninja",
  "binaryDir": "${sourceDir}/build",
  "cacheVariables": {
    "CMAKE_BUILD_TYPE": "Release",
    "BOARD": "XXX"
  }
}
```

```
    }  
},
```

Либо в файле `.\Projects\CMakeLists.txt`:

```
set(BOARD "XXX")
```

Либо любым другим удобным способом.

Прошивка

В меню **Terminal->Run Task** доступны задания по прошивке различными инструментами. (TODO: протестировать)

Отладка

В меню **Run and Debug** доступны варианты отладки различными инструментами.

Порядок действий:

Собираем проект, выбираем вариант отладки, запускаем F5.

Чтобы запитать Target через J-Link, выполнить команду в J-Link Commander

J-Link power on

Или чтобы программатор принял настройку по-умолчанию.

J-Link power on perm

(TODO: проверить все на Linux)

Новый проект:

1. Создать директорию нового проекта в `/Projects`
2. Добавить `CMakeLists.txt` по аналогии с демо проектом. Определить глобальные свойства:

```
MCU - модель микроконтроллера  
MCU_MANUFACTURER - производитель  
CORE - архитектура  
PROJECT - имя проекта  
При необходимости:  
TOOLCHAIN_DIR  
TOOLCHAIN_PREFIX
```

Данные свойства могут подменяться через условный оператор над свойством `BOARD`, если один проект рассчитано запускать на разных платформах.

1. Создать файл графа инициализации `mig_<mcu>.c`. MIG файлы должны быть уникальны для платформы-проекта. Данный файл описывает модули используемые в проекте, их зависимости и конфигурации. Это полное описание системы от ядра контроллера и вектора его прерываний до высокоуровневых драйверов внешних связанных устройств.
*можно адаптировать существующий из другого проекта.
2. Создать `mig_<mcu>.h`, тут экспортируются все дескрипторы модулей задействованных в проекте. По сути это требования бизнес-логики к платформе, какими ресурсами она должна обладать.

```
#ifndef MIG_H_
#define MIG_H_

extern hdl_time_counter_t mod_timer_ms; // само определение будет дано в
mig_<mcu>.c
...

#endif // MIG_H_
```

3. Для удобства создать `app.h`, базовые зависимости бизнес-логики (по желанию).

```
#ifndef APP_H_
#define APP_H_

#include "hdl_iface.h"
#include "mig_<mcu>.h"

void main();

#endif /* APP_H_ */
```

4. Создать файл точку входа проекта `app.c`.

```
#include "app.h"

void main() {
    // включаем необходимые модули
    hdl_enable(&mod_timer_ms);
    while (!hdl_init_complete()) {
        cooperative_scheduler(false);
    }
    // инициализация модулей завершена
    while (1) {
        cooperative_scheduler(false);
        // логика приложения
    }
}
```

`hdl_enable(&mod_timer_ms);` - включает модуль таймера миллисекунд.
`cooperative_scheduler(false)` - работа кооперативного планировщика.

5. Привязка дескрипторов модулей к их определениям в `mig_<mcu>.c`

```
...
const hdl_systick_counter_t mod_systick_counter = {
    .iface = &hdl_systick_counter_iface,
    .dependencies = hdl_module_dependencies(&mod_clock_ahb),
    .config = hdl_module_config(hdl_systick_counter_config_t,
    .phy = (uint32_t)SysTick,
    .period = HDL_SYSTICK_COUNTER_RELOAD,
    #if (HDL_SYSTICK_PRESCALER == 8)
        .clock_src = 0
    #else
        .clock_src = SysTick_CTRL_CLKSOURCE_Msk
    #endif
    ),
    .mod_var = static_malloc(HDL_MODULE_VAR_SIZE)
};

const hdl_time_counter_t mod_systick_timer = {
    .iface = &hdl_time_counter_iface,
    .dependencies = hdl_module_dependencies(&mod_systick_counter,
    &mod_irq_systick),
    .mod_var = static_malloc(HDL_MODULE_VAR_SIZE),
    .obj_var = static_malloc(HDL_TIME_COUNTER_VAR_SIZE),
};
...

extern const hdl_time_counter_t mod_timer_ms
__attribute__((alias ("mod_systick_timer")));
```

В данном примере инженер может подразумевать использование системного таймера для счета миллисекунд, на другой платформе это может быть совершенно другой модуль реализующий интерфейс счетчика времени. Так бизнес логика абстрагируется от аппаратной платформы.